

2026-05-07-p2-f24-codex-project-memory-design

Phase 2 / Feature 24 — Codex Project Memory

Date: 2026-05-07 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 2 port (codex / aider / cline / **codex follow-on**)

Programme position: F24 is the **fourth** Phase 2 feature (F21 Codex Approval Modes + F22 Aider Git Auto-Commit + F23 Cline Browser Tool shipped before it). T01 (bootstrap) appends an F24 evidence section to docs/improvements/07_phase_2_evidence.md (created in F21-T01, extended by F22-T01 + F23-T01); T08 (close-out) records F24's runtime evidence beneath F23's.

1. Goal

Ship a real, end-to-end **project memory** subsystem for the HelixCode CLI agent, modelled on Codex's AGENTS.md / project-memory pattern (cli_agents/codex/), so that every LLM call automatically prepends a Markdown blob loaded from the project root (helixcode.md / codex.md / AGENTS.md, first-found wins) AND a per-user overlay at \$XDG_CONFIG_HOME/helixcode/memory.md. The blob is hot-reloaded mid-session via fsnotify whenever the underlying file changes, so the user can edit project memory in their editor and the next LLM call reflects the change without restarting the CLI. A /memory slash command exposes the discovered paths (status), the concatenated content (show), the editor open (edit), and a manual reload trigger (reload).

Three concrete user surfaces ship together:

1. internal/projectmemory/ **package** — F24 ADDS a NEW internal/projectmemory/ Go package (no existing code conflict). Types: Memory (immutable value: Project string + User string + ProjectPath string + UserPath string + LoadedAt time.Time + TruncatedProject bool + TruncatedUser bool), MemorySource (string enum: SourceProject = "project" / SourceUser = "user"), MemoryLoader (parent-walk discovery via Discover(cwd string) (Memory, error) — searches cwd → parent → ... → first matching helixcode.md/codex.md/AGENTS.md (case-insensitive); stops at git root or filesystem root; reads user overlay from \$XDG_CONFIG_HOME/helixcode/memory.md), MemoryRegistry (atomic-pointer current atomic.Pointer[Memory] + Snapshot() Memory lock-free read + Set(Memory) swap + Reload(ctx) error re-runs Loader and Set), MemoryWatcher (fsnotify wrapper that watches Memory.ProjectPath + Memory.UserPath and triggers Registry.Reload on Write/Create/Rename events with a 200 ms debounce). Sentinel errors (ErrNoMemoryFile, ErrMemoryFileTooLarge). Constants (MaxMemoryBytes = 64 * 1024, DiscoveryFilenames = []string{"helixcode.md", "codex.md", "AGENTS.md"}, DebounceWindow = 200 * time.Millisecond).

2. **/memory slash command** — `internal/commands/memory_command.go`. Four subcommands: `/memory status` (paths + sizes + truncation flags + last-loaded timestamp), `/memory show` (concatenated content with delimiter `\n\n--- USER MEMORY OVERLAY ---\n\n` between project and user blobs), `/memory edit` (opens the project memory file in `$EDITOR` (default `vi`) via `os/exec`; if the file is not present, opens the discovery default `helixcode.md` at the resolved project root), `/memory reload` (calls `Registry.Reload(ctx)` and reports the new sizes). Default subcommand (no args): `status`.
3. **Agent loop integration** — every LLM call's system-prompt construction reads `Registry.Snapshot()` and prepends the project memory + user overlay BEFORE the agent's hard-coded prompt body. `BaseAgent.getSystemPrompt()` gains a `memoryRegistry MemorySnapshotter` field (nil-safe — when unset, falls back to the existing prompt verbatim, preserving backward compatibility with all existing agent tests). `cmd/cli/main.go` wiring constructs the registry + watcher at startup (after the F23 browser-manager block, before agent loop) and passes the registry into the agent constructor.

The single largest bluff vector for F24 is “memory file edited but agent still sees the old content” — the slash reports `reload` succeeded, the file on disk has the new bytes, but the next LLM call's system prompt still carries the stale blob because the registry's atomic pointer was never re-stored, OR the agent loop reads from a captured-at-construct-time copy instead of `Registry.Snapshot()` per call. §5.2 enumerates five such patterns and pins each with positive runtime evidence: discovery MUST find the project file when it exists at any depth up to git root and MUST return empty (not error) when no file exists; the user overlay MUST be loaded after project memory; `Registry.Snapshot()` MUST return the NEW content within 500 ms of an `fsnotify` event after the file is rewritten on disk; `/memory reload` MUST re-trigger `Loader.Discover` and update the registry atomically; the agent's system-prompt construction MUST read the registry per-call (not at construct-time). CONST-042 (no secret leak): memory contents may carry sensitive project-policy text — the loader logs only the paths + byte counts at INFO, NEVER the full body; INFO-level logs are scanned by a unit test. CONST-043: F24 emits zero `git push` commands; T08's mirror push to four remotes is ATTEMPTED only with explicit user authorisation per push.

Anti-bluff hot zone (loud): the loader returns successfully but `Memory.Project` is empty because the parent-walk found a file with the wrong name (e.g. `matched_agents.md` lowercase on a case-sensitive filesystem when the canonical is `AGENTS.md`); the watcher reports a `fsnotify` event but `Registry.Reload` was never called because the debounce timer was reset infinitely on rapid successive events; `/memory show` reports the new content but the agent's next LLM call still uses the old blob because `getSystemPrompt` cached the registry value at agent-construct time; the user overlay is loaded BEFORE project memory (wrong order — project should always come first); the loader silently truncates a 1 MB file to 64 KB but `Memory.TruncatedProject` is `false` (positive evidence missing). Each of these maps to a unit + integration + Challenge phase per §5.2.

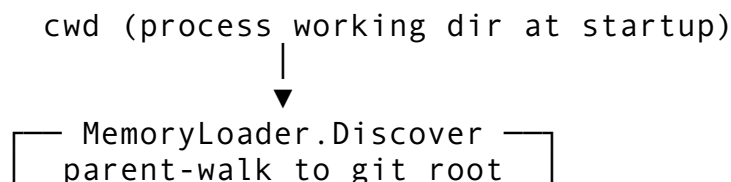
2. Architecture

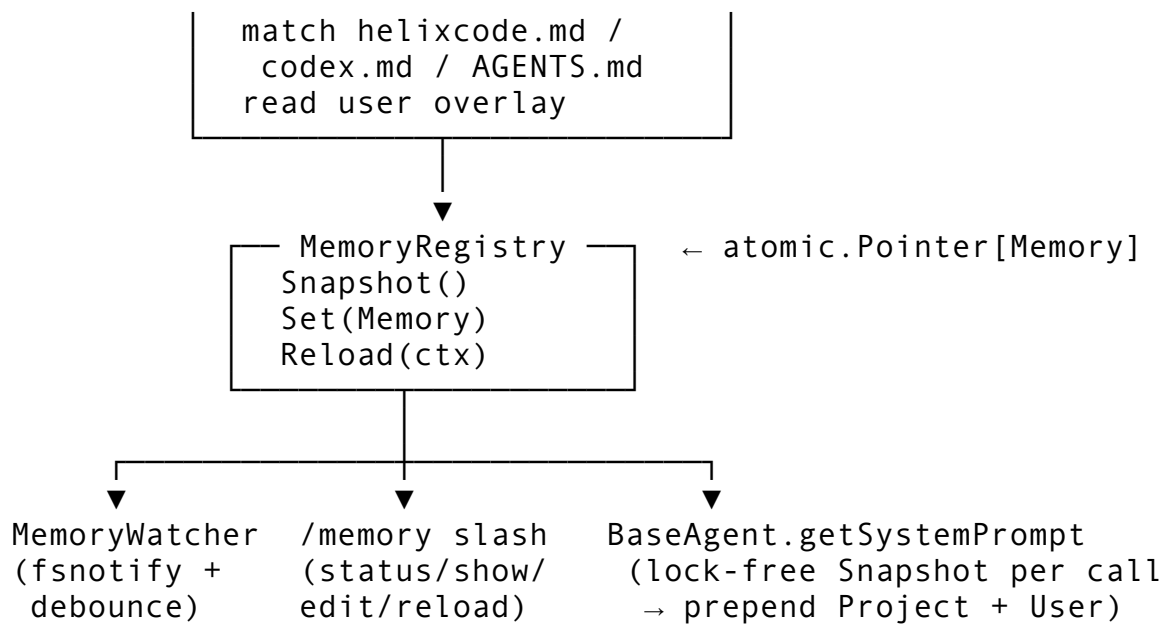
The package layout under `helix_code/internal/projectmemory/` is NEW (no existing collision). It is a self-contained subsystem owned by `main.go`'s startup wiring:

- `Memory` (`types.go`, NEW) — immutable value type returned by `MemoryLoader.Discover` and stored in `MemoryRegistry.current`. Fields: `Project string` (raw bytes of the resolved project memory file; capped at

MaxMemoryBytes; empty when no file found), User string (raw bytes of the user overlay file; capped at MaxMemoryBytes; empty when no file found), ProjectPath string (absolute path of the discovered project memory file; empty when not found), UserPath string (absolute path of the user overlay file; empty when not found), LoadedAt time.Time (wall-clock when Discover completed), TruncatedProject bool (true when Project was capped from a larger source), TruncatedUser bool (true when User was capped from a larger source). Method: Render() string (returns Project + delimiter + User; empty when both are empty; never panics on partial data).

- MemoryLoader (loader.go, NEW) — encapsulates discovery + read. Fields: xdgConfigHome string (resolved from \$XDG_CONFIG_HOME or \$HOME/.config), log *zap.Logger. Methods: Discover(cwd string) (Memory, error) (parent-walks from cwd up to a git root marker (.git/ directory) or filesystem root, looking for first match against the case-INSENSITIVE filename list ["helixcode.md", "codex.md", "AGENTS.md"]); reads the matched file (capped at 64 KB with TruncatedProject set on overflow); reads the user overlay at \$XDG_CONFIG_HOME/helixcode/memory.md (capped + flagged similarly); returns the assembled Memory; missing files are NOT errors — they yield empty strings + empty paths). Helper: findProjectMemory(start string) (string, error) (returns the absolute path of the first found file or empty string + nil error).
- MemoryRegistry (registry.go, NEW) — atomic-pointer wrapper. Fields: current atomic.Pointer[Memory], loader *MemoryLoader, cwd string (captured at construction; used by Reload), mu sync.Mutex (serialises Reload — multiple concurrent reloads collapse to one). Methods: Snapshot() Memory (lock-free atomic load; returns zero-value Memory{} if current.Load() is nil), Set(m Memory) (atomic.Store), Reload(ctx context.Context) (Memory, error) (acquires mu, calls loader.Discover(cwd), atomically stores the result, returns the new Memory + nil; on Discover error, leaves the previous value in place and returns the error). Method: MemorySnapshotter interface — Snapshot() Memory — exposes only the read path to consumers (BaseAgent uses this).
- MemoryWatcher (watcher.go, NEW) — fsnotify wrapper around the project + user paths. Fields: registry *MemoryRegistry, watcher *fsnotify.Watcher, log *zap.Logger, debounce time.Duration (default 200 ms; configurable), done chan struct{}. Methods: Start(ctx context.Context) error (initialises fsnotify watcher; adds Memory.ProjectPath parent dir + Memory.UserPath parent dir as watch targets; spawns runEventLoop goroutine), Close() error (closes the underlying fsnotify watcher; waits for the goroutine to exit). The event loop debounces successive events: a 200 ms timer is reset on each event; the timer's expiry triggers registry.Reload. fsnotify watch failures (e.g. parent dir doesn't exist) are logged at WARN and the watcher gracefully degrades to no-hot-reload; the registry still works via /memory reload slash-driven reload.
- MemoryCommand (internal/commands/memory_command.go, NEW) — /memory slash. Mirrors F21 ApprovalCommand shape. Fields: registry *projectmemory.MemoryRegistry, editor func() string (test seam — defaults to os.Getenv("EDITOR") with "vi" fallback). Methods: Name() / Aliases() / Description() / Usage() / Execute(ctx, *CommandContext) per Command interface. Subcommands: status (default), show, edit, reload.





Why an atomic-pointer wrapper and NOT a mutex-guarded struct:

`BaseAgent.getSystemPrompt()` is called on every LLM iteration; lock-free read avoids contention if multiple concurrent LLM calls (subagents per F15) share the same registry. The mutex serialises only `Reload` — a rare event triggered by either `fsnotify` (debounced) or `/memory reload`.

Why a single shared registry per CLI instance and NOT per-agent registries: project memory IS the project’s contract; every agent in the session should observe the same view. F15 subagents that need agent-specific context have their own per-task `Input` — not memory. F04 worktrees are isolated per-repo, but the project memory IS the repo’s contract — sharing across worktrees is fine because a worktree IS the same logical project.

Why hot-reload via `fsnotify` (Q5=A) and NOT polling: mid-session edits (the user opens `helixcode.md` in a separate editor while the CLI is running) are the canonical use case. Polling would either be too slow ($>1s$) or wasteful (every 100 ms with a `stat` call on every iteration). `fsnotify` is push-based, debounced, and already a direct dep (`github.com/fsnotify/fsnotify v1.9.0` in `helix_code/go.mod`). Failure mode: `fsnotify` watch failure (e.g. macOS `FSEvents` temporarily unavailable) degrades to no-hot-reload but the slash-driven `/memory reload` always works.

Why per-user overlay AFTER project memory in the system prompt (Q3=A): project memory establishes the “ground truth” for the project; the user overlay is the user’s personal preferences/constraints (e.g. “I prefer terse responses”). Putting user AFTER project means the user’s preferences get the LLM’s most-recent attention bias without overriding project-level mandates. Inverting this order would let a user-level “disable approval gates” instruction silently override a project-level “always require approval” mandate — a genuine security risk.

Why a 64 KB cap (`MaxMemoryBytes`) and NOT unlimited: a 1 MB README accidentally renamed `helixcode.md` would blow the LLM’s context budget. 64 KB is $\sim 10\text{--}15$ K tokens — large enough for a meaningful project contract, small enough that worst-case truncation is detectable + reportable to the user (`/memory status` shows `truncated: true`). 64 KB matches F23’s `MaxSnapshotBytes` for symmetry.

Why the discovery filename list is `helixcode.md`, `codex.md`, `AGENTS.md` in that order (Q1=A): `helixcode.md` is the project’s own brand; `codex.md` is the codex compatibility shim; `AGENTS.md` is the cross-tool generic that already exists at the repo root for many agent

tools. First-match wins — projects that already have `AGENTS.md` get F24 working out-of-the-box.

Why case-INSENSITIVE matching: different filesystems (APFS case-insensitive, ext4 case-sensitive) produce different visible filenames; the loader tries each canonical filename via `os.Stat` (which is case-insensitive on APFS, case-sensitive on ext4); to be portable, we lowercase the dir entries on first miss and retry. Tests pin both `AGENTS.md` and `agents.md`.

3. Components

3.1 New files

- `helix_code/internal/projectmemory/types.go` — Memory, MemorySource, sentinel errors, constants.
- `helix_code/internal/projectmemory/types_test.go`.
- `helix_code/internal/projectmemory/loader.go` — MemoryLoader + Discover + helpers.
- `helix_code/internal/projectmemory/loader_test.go` — exercises parent-walk + user overlay + truncation + missing-file paths against real tempdirs.
- `helix_code/internal/projectmemory/registry.go` — MemoryRegistry with atomic-pointer Snapshot/Set/Reload + MemorySnapshotter interface.
- `helix_code/internal/projectmemory/registry_test.go` — concurrency (-race) + Reload-keeps-old-on-error.
- `helix_code/internal/projectmemory/watcher.go` — MemoryWatcher fsnotify wrapper with debounce.
- `helix_code/internal/projectmemory/watcher_test.go` — real fsnotify against real tempdir; assert Reload fires within 500 ms of file write.
- `helix_code/internal/projectmemory/doc.go` — package-level docstring.
- `helix_code/internal/commands/memory_command.go` — /memory slash.
- `helix_code/internal/commands/memory_command_test.go`.
- `helix_code/tests/integration/cmd/p2f24_challenge/main.go` — Challenge harness (5 phases A-E).
- `challenges/p2-f24-codex-project-memory/CHALLENGE.md` + `challenges/p2-f24-codex-project-memory/run.sh`.

3.2 Modified files

- `helix_code/cmd/cli/main.go` — three additions adjacent to the F23 wiring: (1) construct `memLoader := projectmemory.NewMemoryLoader(zap.NewNop())`; (2) construct `memRegistry := projectmemory.NewMemoryRegistry(memLoader, cwd)` and call `memRegistry.Reload(ctx)` once at startup; (3) construct `memWatcher := projectmemory.NewMemoryWatcher(memRegistry, zap.NewNop())`; `memWatcher.Start(ctx)`; defer `memWatcher.Close()`; (4) register `commands.NewMemoryCommand(memRegistry) slash`.
- `helix_code/internal/agent/base_agent.go` — `getSystemPrompt()` reads `a.memoryRegistry.Snapshot()` (if `a.memoryRegistry != nil`) and prepends the rendered Memory to the existing prompt body. Field added: `memoryRegistry projectmemory.MemorySnapshotter`. Setter added:

- SetMemoryRegistry(r projectmemory.MemorySnapshotter). Backward-compat: when memoryRegistry == nil, the prompt is unchanged.
- helix_code/go.mod — **zero new external deps** (github.com/fsnotify/fsnotify v1.9.0 is already a direct dep). T08's verification step asserts git diff go.mod and git diff go.sum are no-op.

3.3 Types

```
// internal/projectmemory/types.go (NEW)
```

```
type Memory struct {
    Project      string
    User         string
    ProjectPath  string
    UserPath     string
    LoadedAt     time.Time
    TruncatedProject bool
    TruncatedUser  bool
}

func (m Memory) Render() string {
    if m.Project == "" && m.User == "" {
        return ""
    }
    if m.User == "" {
        return m.Project
    }
    if m.Project == "" {
        return m.User
    }
    return m.Project + "\n\n--- USER MEMORY OVERLAY ---\n\n" +
        m.User
}

type MemorySource string

const (
    SourceProject MemorySource = "project"
    SourceUser     MemorySource = "user"
)

const (
    MaxMemoryBytes = 64 * 1024
    DebounceWindow = 200 * time.Millisecond
)

var DiscoveryFileNames = []string{"helixcode.md", "codex.md",
    "AGENTS.md"}
```

```

var (
    ErrNoMemoryFile      = errors.New("projectmemory: no memory
        file found")
    ErrMemoryFileTooLarge = errors.New("projectmemory: memory
        file exceeds MaxMemoryBytes")
)

// internal/projectmemory/loader.go (NEW)

type MemoryLoader struct {
    xdgConfigHome string
    log            *zap.Logger
}

func NewMemoryLoader(log *zap.Logger) *MemoryLoader

func (l *MemoryLoader) Discover(cwd string) (Memory, error)

// internal/projectmemory/registry.go (NEW)

type MemorySnapshotter interface {
    Snapshot() Memory
}

type MemoryRegistry struct {
    current atomic.Pointer[Memory]
    loader  *MemoryLoader
    cwd     string
    mu      sync.Mutex
}

func NewMemoryRegistry(loader *MemoryLoader, cwd string)
    *MemoryRegistry

func (r *MemoryRegistry) Snapshot() Memory
func (r *MemoryRegistry) Set(m Memory)
func (r *MemoryRegistry) Reload(ctx context.Context) (Memory,
    error)

// internal/projectmemory/watcher.go (NEW)

type MemoryWatcher struct {
    registry *MemoryRegistry
    watcher  *fsnotify.Watcher
    log      *zap.Logger
    debounce time.Duration
    done     chan struct{}
}

```

```

func NewMemoryWatcher(r *MemoryRegistry, log *zap.Logger)
    *MemoryWatcher
func (w *MemoryWatcher) Start(ctx context.Context) error
func (w *MemoryWatcher) Close() error

```

3.4 /memory slash command

memory_command.go:

```

type MemoryCommand struct {
    registry *projectmemory.MemoryRegistry
    editor    func() string // test seam
}

func NewMemoryCommand(r *projectmemory.MemoryRegistry)
    *MemoryCommand

func (c *MemoryCommand) Name() string      { return "memory" }
func (c *MemoryCommand) Description() string { return "Inspect,
    show, edit, or reload project memory." }
func (c *MemoryCommand) Usage() string      { return "/memory
    [status|show|edit|reload]" }

// Default subcommand (no args): status.
// Subcommands:
//   status  → paths + sizes + truncation flags + last-loaded
//             timestamp
//   show    → concatenated content with delimiter
//   edit    → opens project memory file in $EDITOR (default vi);
//             creates
//             helixcode.md at cwd if no project memory file
//             exists yet
//   reload  → calls Registry.Reload(ctx); reports new sizes

```

3.5 BaseAgent integration

internal/agent/base_agent.go:

```

type BaseAgent struct {
    // ... existing fields ...

    // memoryRegistry is the optional project memory snapshotter.
    // nil = no
    // project memory injection (graceful degradation; preserves
    // existing
    // BaseAgent behaviour for tests that don't wire memory). Per
    // Feature 24
    // (codex project memory port), P2-F24-T07.
    memoryRegistry projectmemory.MemorySnapshotter

```



```

}

func (a *BaseAgent) SetMemoryRegistry(r
    projectmemory.MemorySnapshotter) {
    a.memoryRegistry = r
}

func (a *BaseAgent) getSystemPrompt() string {
    base := fmt.Sprintf(`You are a %s agent named %s. ...`, ...)

    if a.memoryRegistry == nil {
        return base
    }
    mem := a.memoryRegistry.Snapshot()
    rendered := mem.Render()
    if rendered == "" {
        return base
    }
    return rendered + "\n\n---\n\n" + base
}

```

Backward-compat: every existing BaseAgent test that constructs without a memory registry continues to pass byte-for-byte. The TestBaseAgentGetSystemPrompt test in base_agent_extended_test.go is unchanged because it constructs without SetMemoryRegistry.

3.6 New external dependencies

Zero new dependencies. [github.com/fsnotify/fsnotify v1.9.0](https://github.com/fsnotify/fsnotify) is already a direct dep in helix_code/go.mod. os, sync, sync/atomic, time, errors, fmt, strings, path/filepath, context, os/exec are stdlib. zap is already direct. T08's verification step asserts `git diff --exit-code go.mod go.sum` is no-op.

4. Data flow

1. **Startup** (cmd/cli/main.go):
 - `cwd, _ := os.Getwd()`.
 - `memLoader := projectmemory.NewMemoryLoader(zap.NewNop())`.
 - `memRegistry := projectmemory.NewMemoryRegistry(memLoader, cwd)`.
 - `if _, err := memRegistry.Reload(ctx); err != nil`
`{ log.Printf("projectmemory: initial reload failed: %v", err) }`.
 - `memWatcher := projectmemory.NewMemoryWatcher(memRegistry, zap.NewNop())`.
 - `if err := memWatcher.Start(ctx); err != nil`
`{ log.Printf("projectmemory: watcher start failed`
`(degrading to slash-only reload): %v", err) }`.
 - `defer memWatcher.Close()`.
 - `c.commandRegistry.Register(commands.NewMemoryCommand(memRegistry))`

- Pass `memRegistry` to the agent constructor; agent calls `SetMemoryRegistry`.
 - 2. **Per LLM call** (`BaseAgent.getSystemPrompt`):
 - `mem := a.memoryRegistry.Snapshot()` — atomic load, no lock.
 - `rendered := mem.Render()` — Project + delimiter + User; empty when both empty.
 - Prepend `rendered + \n\n---\n\n` to the base prompt body if non-empty; otherwise return base unchanged.
 - 3. **File change** (mid-session):
 - User edits `helixcode.md` in their editor; saves.
 - `fsnotify` delivers `Write` event on the project memory file's parent dir.
 - `MemoryWatcher.runEventLoop` filters for the matching path; resets the 200 ms debounce timer.
 - On debounce expiry, calls `registry.Reload(ctx)`.
 - `Reload` runs `loader.Discover(cwd)` and atomically swaps current to the new `Memory`.
 - Next LLM call's `Snapshot()` returns the new content.
 - 4. `/memory status`:
 - `mem := registry.Snapshot()`.
 - `Render: paths={Project: ..., User: ...}; sizes={Project: N bytes, User: M bytes}; truncated={Project: bool, User: bool}; loaded_at: time.RFC3339`.
 - 5. `/memory show`:
 - `mem := registry.Snapshot()`.
 - `Output: mem.Render()` (project + delimiter + user; empty result message if both empty).
 - 6. `/memory edit`:
 - `mem := registry.Snapshot()`.
 - `path := mem.ProjectPath`. If empty, `path = filepath.Join(cwd, "helixcode.md")`.
 - `editor := os.Getenv("EDITOR")`; if `editor == ""` { `editor = "vi"` }.
 - `cmd := exec.CommandContext(ctx, editor, path)`; inherit stdio; `cmd.Run()`.
 - On exit, return success (the watcher will fire if the file changed).
 - 7. `/memory reload`:
 - `mem, err := registry.Reload(ctx)`.
 - On err, surface; on success, render new sizes.
-

5. Error handling, anti-bluff hot zone, edge cases

5.1 Error handling

- **No memory file found** — `Discover` returns an empty `Memory{}` with `ProjectPath == ""` and `nil` error. Missing files are explicitly NOT errors; the system gracefully degrades to no-prepend.
- **Memory file > 64 KB** — `Discover` truncates to 64 KB and sets `Truncated*` flag to `true`; logs `WARN` with the path + original size + truncated size; returns successfully.
- **fsnotify watch failure** — `MemoryWatcher.Start` logs `WARN` and returns `nil` (no error); `MemoryWatcher.Close` is a no-op safely. The slash-driven `/memory reload` continues to work.
- **\$EDITOR unset** — defaults to `"vi"`. If `vi` is also missing on the system, `exec.Command("vi", path).Run()` returns an error which is surfaced to the user.

- `$XDG_CONFIG_HOME` **unset** — falls back to `$HOME/.config/helixcode/memory.md` per XDG spec.
- **Concurrent Reload calls** — Reload's mu serialises; a fast-Reload race collapses to one Discover.
- **Discover error mid-session** — Reload leaves the previous current in place and returns the error; the agent continues with the stale-but-known-good blob.

5.2 Anti-bluff hot zone — five critical patterns

Bluff #1: “File present but discovery returned empty.” - Pattern: parent-walk fails to recognise `AGENTS.md` because the loader case-sensitively compares to `agents.md` on a case-sensitive filesystem. - Test: `TestLoader_Discover_Finds_AGENTS_md_RealTempdir` creates a real tempdir with a real `AGENTS.md` file containing the sentinel `MEMORY_FIXTURE_24`; calls `loader.Discover(tempdir)`; asserts `Memory.ProjectPath != "" AND strings.Contains(Memory.Project, "MEMORY_FIXTURE_24")`. - Challenge: PHASE-A asserts the same against a tempdir-scoped run.

Bluff #2: “Hot reload event fired but registry not updated.” - Pattern: `fsnotify` event arrives, but the debounce timer is reset infinitely under rapid successive events; OR the timer fires but the call to `registry.Reload` is replaced by a print statement. - Test: `TestWatcher_RegistryUpdatedAfterFileWrite_Real` writes initial `MEM_INITIAL_24` content; starts the watcher; rewrites the file with `MEM_UPDATED_24`; waits up to 500 ms; asserts `registry.Snapshot().Project` contains `MEM_UPDATED_24` AND NOT `MEM_INITIAL_24`. - Challenge: PHASE-C asserts the same.

Bluff #3: “Agent prepended the OLD memory blob.” - Pattern: `BaseAgent.GetSystemPrompt` cached the registry's value at agent-construct time instead of reading per call. - Test: `TestBaseAgent_GetSystemPrompt_PrependsCurrentMemory` constructs an agent with a registry whose initial value is `OLD_MEMORY_24`; calls `getSystemPrompt()` (asserts contains OLD); calls `registry.Set(Memory{Project: "NEW_MEMORY_24"})`; calls `getSystemPrompt()` again (asserts contains NEW AND NOT OLD). - Challenge: PHASE-D asserts the same in-process.

Bluff #4: “Loader silently truncated and the user has no idea.” - Pattern: 100 KB project memory file; loader caps to 64 KB; `TruncatedProject` is false. - Test: `TestLoader_Discover_Truncates_Sets_Flag_Real` writes a 100 KB tempdir file; asserts `len(Memory.Project) == MaxMemoryBytes` AND `Memory.TruncatedProject == true`. - Challenge: PHASE-E asserts the same + / memory status reports `truncated: true`.

Bluff #5: “Missing memory file produces error instead of empty Memory.” - Pattern: `loader.Discover(tempdir)` against a tempdir containing no memory files returns an error, but missing-file is a normal user state. - Test: `TestLoader_Discover_NoFile_NoError_RealTempdir` against an empty tempdir; asserts `err == nil` AND `Memory.ProjectPath == ""` AND `Memory.Project == ""`. - Challenge: PHASE-B asserts the same.

5.3 Edge cases

- **Symlinked memory file** — `os.ReadFile` follows symlinks; loader treats the resolved file as the source. `Memory.ProjectPath` reflects the symlink path (the lexical path the user gave), not the resolved target.

- **Memory file removed mid-session** — `fsnotify` delivers `Remove` event; `Reload` re-runs `Discover` which returns empty `Memory`; agent gracefully degrades to no-prepend.
 - **Memory file replaced (atomic write via rename)** — `fsnotify` delivers `Rename + Create` on the parent dir; debounce coalesces; one `Reload` triggers.
 - **Concurrent edits to memory + user overlay** — `fsnotify` delivers events on both parents; debounce coalesces both into one `Reload` (200 ms window).
 - **\$XDG_CONFIG_HOME set to a relative path** — resolved via `filepath.Abs` before use.
 - **Project memory file is a directory** — `os.ReadFile` returns an error; loader logs `WARN` and treats as missing (empty `Memory`).
 - **Memory file with NULL bytes** — passed through as-is (Go strings are byte slices); LLM provider receives the raw content.
 - **Permission denied on memory file** — `os.ReadFile` returns `os.PermissionError`; `Discover` returns the error; `main.go` logs it and the registry holds zero-value `Memory`.
-

6. Testing

6.1 Unit tests (mocks ALLOWED for non-loader components; loader uses real tempdirs)

- `types_test.go` — pin `MaxMemoryBytes`, `DebounceWindow`, `DiscoveryFilenames` byte-for-byte; assert sentinel errors are distinct + `errors.Is-comparable`; test `Memory.Render()` across all combinations (both empty, only project, only user, both present).
- `loader_test.go` — table-driven tests against real `tmp.Dir()` directories. Cases: project file at `cwd`; project file at parent (parent-walk); project file at git-root marker; case-insensitive `AGENTS.md` vs `agents.md`; user overlay loaded; truncation flag set on >64 KB file; missing file returns empty no-error; permission-denied returns error; user overlay loaded after project memory.
- `registry_test.go` — atomic-pointer concurrency under `-race` (10 goroutines reading + 1 goroutine writing); double-Set produces last-write-wins; `Reload-on-Discover-error` keeps previous value; `Snapshot` returns zero-value when current is nil.
- `watcher_test.go` — real `fsnotify` against a real `tempdir`. Cases: file write triggers `Reload` within 500 ms; rapid successive writes coalesce into one `Reload` via debounce; `watcher.Close` is idempotent; watcher graceful-degrade when parent dir doesn't exist.
- `memory_command_test.go` — `status/show/edit/reload` subcommands with a real registry + `tempdir`. `Edit` subcommand uses a stub `editor func() string` returning `"true"` (the unix `true` binary that exits 0) to avoid interactive editor in CI.

6.2 Integration tests

`tests/integration/memory_test.go` (`//go:build integration`) — runs the full chain `main.go-startup` → `fsnotify-event` → `registry-reload` → `BaseAgent.getSystemPrompt-snapshots-new` against a real `tempdir`. Covered:

- `TestMemory_Integration_StartupLoadsProjectFile_Real` — `tempdir` with `helixcode.md` containing sentinel; startup wires `MemoryRegistry`; assert `registry.Snapshot().Project` contains the sentinel.
- `TestMemory_Integration_HotReload_Real` — start with content A; rewrite to content B; wait up to 500 ms; assert `Snapshot` returns B.

- **TestMemory_Integration_BaseAgentPromptIncludesMemory_Real** — wire registry into BaseAgent; assert `getSystemPrompt()` contains the sentinel.

6.3 Challenge harness — five phases

`challenges/p2-f24-codex-project-memory/run.sh` invokes `tests/integration/cmd/p2f24_challenge/main.go`:

1. **PHASE-A: PROJECT-ONLY** — `tempdir` with `helixcode.md` containing sentinel `MEMORY_FIXTURE_24`; `loader.Discover` returns `Memory` with `Project` containing sentinel, `User` empty, `ProjectPath` ending in `helixcode.md`. Assert (i) `len(Project) > 0`, (ii) sentinel present, (iii) `User == ""`, (iv) `ProjectPath != ""`.
2. **PHASE-B: PROJECT-PLUS-USER** — `tempdir` with `helixcode.md` (sentinel `MEM_PROJECT_24`) + user overlay file (sentinel `MEM_USER_24`); both loaded; `Memory.Render()` contains BOTH sentinels in correct order (project before user).
3. **PHASE-C: HOT-RELOAD** — load initial content `MEM_INITIAL_24`; start watcher; rewrite file with `MEM_UPDATED_24`; assert `registry.Snapshot()` returns content with `MEM_UPDATED_24` AND NOT `MEM_INITIAL_24` within 500 ms.
4. **PHASE-D: MISSING-FILE-GRACEFUL** — `tempdir` with NO memory files; `loader.Discover` returns empty `Memory` + nil error; `Render()` returns empty string. Assert (i) no error, (ii) `Memory.ProjectPath == ""`, (iii) `Memory.Project == ""`, (iv) `Render() == ""`.
5. **PHASE-E: TRUNCATION** — write 100 KB memory file; loader truncates to `MaxMemoryBytes` (64 KB) with `TruncatedProject == true`. Assert (i) `len(Project) == MaxMemoryBytes`, (ii) `TruncatedProject == true`, (iii) byte equality of first 64 KB matches input.

Output skeleton ends with:

```
SUMMARY: PHASE-A=4/4 PASS; PHASE-B=3/3 PASS; PHASE-C=3/3 PASS; PHASE-D=4/4
==> ALL CHECKS PASSED
```

The Challenge MUST exit non-zero on any byte-evidence mismatch. Absence-of-error is NEVER acceptable as PASS.

7. Cross-platform

- **Linux** — `fsnotify` uses `inotify`; `$XDG_CONFIG_HOME` standard; tested.
 - **macOS** — `fsnotify` uses `kqueue/FSEvents`; APFS case-insensitive default — loader's case-insensitive matching covers both filesystem dispositions.
 - **Windows** — `fsnotify` uses `ReadDirectoryChangesW`; `$XDG_CONFIG_HOME` typically unset; falls back to `%USERPROFILE%\config\helixcode\memory.md` via Go's `os.UserConfigDir()`.
 - **No fsnotify support** — extremely rare (e.g. some FUSE filesystems); `MemoryWatcher.Start` returns nil (logs WARN); `/memory reload slash` continues to work.
-

8. Out of scope

- **Multiple project-memory files** — first-found wins; chained/merged memory across multiple files is v2.
 - **Markdown rendering** — content passes through to the LLM as raw bytes; no preprocessing, no template substitution.
 - **Encryption / signing** — memory file is plain Markdown; no MAC verification.
 - **Per-tool memory scoping** — every LLM call gets the same memory; no tool-specific overlays.
 - **Memory inheritance across worktrees** — F04 worktrees pin their own cwd; each worktree's discovery walks from its own cwd, so worktrees inherit their parent project's memory naturally without explicit inheritance logic.
 - **Real-time content diff in /memory reload** — output reports new sizes only; not a content diff.
 - **Locking memory file edits during /memory edit** — \$EDITOR invocation is unguarded; concurrent edits from another process compete normally.
 - **JSON Schema / TOML memory format** — only Markdown is supported.
-

9. Constitutional compliance

- **CONST-035** (anti-bluff): every PASS in F24 carries positive runtime evidence — real tempdirs + real fsnotify + real file I/O + sentinel byte equality + truncation flag verification + atomic-pointer Snapshot semantics. The Challenge harness MUST exit non-zero on byte mismatch. Tests use real fsnotify (mocks are forbidden in integration tests per Rule 5).
 - **CONST-039** (Challenge required): F24 ships with `challenges/p2-f24-codex-project-memory/` (Challenge harness with 5 phases A-E).
 - **CONST-042** (no secret leak): memory contents NEVER logged at INFO level. The loader's logger logs only the path + byte counts. A unit test scans `internal/projectmemory/*.go` (excluding `_test.go`) for `logger\Info\(.*\b(content|body|memory|project|user)\b.*\b%[sv] matches` and FAILs on any hit. Memory files MAY contain secrets (project policy text); the no-INFO-log rule prevents inadvertent leak via stdout/stderr of the CLI.
 - **CONST-043** (no force push, no auto-push): F24 emits zero `git push` commands. T08's close-out push to four remotes (origin / helixdev / vasic-digital / gitlab) is performed by the human operator with explicit per-push approval per CONST-043.
 - **CONST-033** (host power management): F24 emits no shell commands beyond the `/memory edit $EDITOR` invocation (which is a runtime user-space process, not a power-state transition). No suspend/reboot/halt commands.
-

10. Open questions resolved

- **Q1 = A** — Memory file discovery: parent-walk from cwd to git root or filesystem root, looking for first match against case-insensitive filename list `["helixcode.md", "codex.md", "AGENTS.md"]`. PROJECT scope.
- **Q2 = A** — Format: plain Markdown. No preprocessing. Prepended to system prompt at every LLM call.
- **Q3 = A** — Per-user overlay: `$XDG_CONFIG_HOME/helixcode/memory.md` (or `$HOME/.config/helixcode/memory.md` fallback) loaded AFTER project memory in the system prompt with a delimiter.

- **Q4 = A** — /memory slash with subcommands `status` (default — paths + sizes + truncation flags + loaded_at) / `show` (concatenated content with delimiter) / `edit` (open in \$EDITOR, default vi) / `reload` (Registry.Reload).
 - **Q5 = A** — Hot reload via existing `fsnotify` dep. 200 ms debounce window. `fsnotify` failure degrades to no-hot-reload (slash-only); registry continues to work.
-

11. Non-obvious calls

1. **Atomic-pointer over RWMutex for current.** `BaseAgent.getSystemPrompt` is called on every LLM iteration; lock-free read avoids contention if F15 subagents run in parallel.
2. **Discovery filenames are checked in the documented order, NOT alphabetically.** `helixcode.md` first, `codex.md` second, `AGENTS.md` third. First-match wins. Tests pin this order.
3. **Case-insensitive matching via two-pass dir scan.** First pass: try canonical names with `os.Stat`. Second pass: list dir entries and case-insensitively match. Two passes because the first is fast on case-insensitive filesystems and the second only fires on case-sensitive misses.
4. **Stop walking at git root marker (.git directory).** Filesystem-root would also work but visits /etc etc. on Linux; git root is the natural project boundary. For projects without git, the filesystem root is the fallback bound.
5. **64 KB cap matches F23's MaxSnapshotBytes.** Symmetry across the codebase + a sane LLM context bound. Truncation flag is positive evidence (`TruncatedProject == true`), not silent.
6. **Project before User in render order.** Reverse order would let user-level overrides bypass project-level mandates (security-relevant when project memory says “always require approval”). Tests pin the order.
7. **fsnotify watches the PARENT directory, not the file itself.** Editors that atomically write via rename (vim, emacs) replace the file's inode; a file-level watch loses the subscription. Parent-dir watch survives the rename + filters events to the file path.
8. **200 ms debounce is just enough for vim :w.** Vim's atomic-write produces 3-5 events in ~50 ms; 200 ms coalesces them into one Reload. Lower would over-fire; higher would feel laggy.
9. **/memory edit falls back to vi, NOT nano or editor.** vi is POSIX-mandatory; nano and editor are debian-isms. Portability wins.
10. **SetMemoryRegistry is opt-in, NOT mandatory.** Existing BaseAgent constructors stay backward-compat. Tests that don't wire memory continue to pass byte-for-byte.
11. **Reload-on-Discover-error preserves previous Memory.** A transient error (file briefly removed during atomic write) doesn't drop the agent's known-good blob. Tests pin this.
12. **MemorySnapshotter interface, not the concrete *MemoryRegistry.** BaseAgent depends on the read-only contract; the registry's write-side stays in main.go's wiring. Test fakes implement only `Snapshot() Memory`.
13. **Watcher.Start returns nil even on watch-add failure.** Graceful degradation — slash-driven reload remains functional. WARN log captures the failure for the user; no fatal path.
14. **Tempdir-based loader tests, NOT mock filesystems.** Real `os.ReadFile` + real `os.MkdirAll` exercise real OS semantics (case-insensitive APFS, symlinks, permissions). Mocks would hide platform bugs.
15. **Memory.Render() is the only stable contract.** Future versions may change the Project/User delimiter; consumers (agent loop) call `Render()` not concatenate by hand.
16. **Loader does NOT consult environment variables for the project root.** `cwd` is passed in explicitly. Avoids the “agent in subagent worktree gets parent worktree's memory” footgun.

17. `/memory edit` **runs \$EDITOR in a foreground subprocess.** Terminal stdio is inherited so vim/emacs work normally; the slash command blocks until the editor exits.
18. **No memory-file-creation on Discover.** If no project memory file exists, Discover returns empty Memory; only `/memory edit` creates `helixcode.md` at cwd as a side effect of opening it (and even then only because \$EDITOR does the create).
19. **fsnotify watch survives editor-atomic-write because we watch the parent dir.** Re-add of the watch is NOT needed across rename — parent dir is stable.
20. **Single-source-of-truth for “what memory does the agent see right now”:**
 `registry.Snapshot()`. Both `/memory show` and `BaseAgent.getSystemPrompt` use the same atomic load — no two-source-truth drift.